

PLAYER 1 

HIGHSCORE 2500

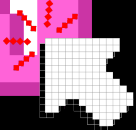
 PLAYER 2

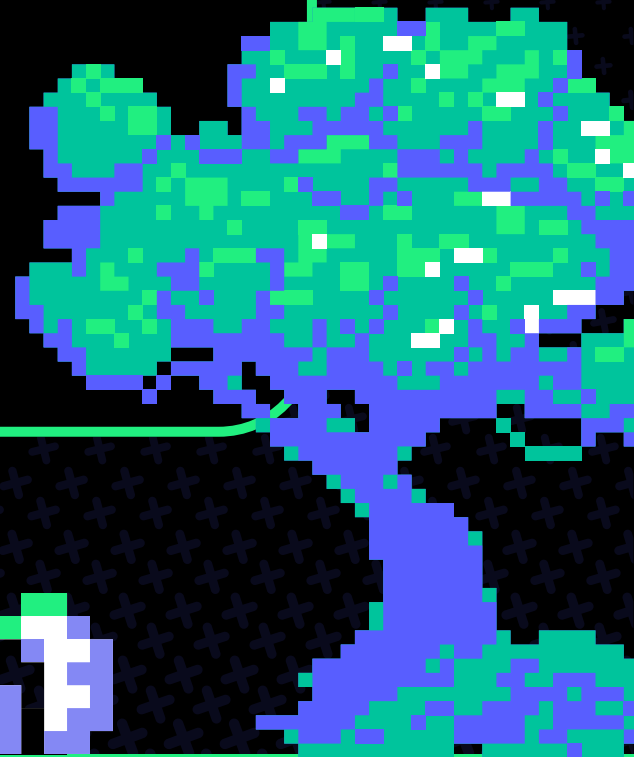
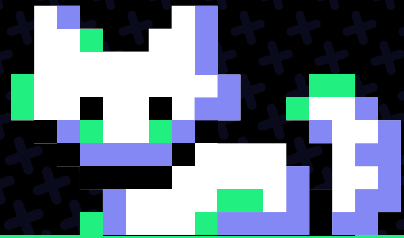
C-MINIPROJECT

◆ PROJECT NAME: THE C GAME HUB

START

MENU

SIGN IN 



TEAM MEMBERS

TANISK SINHA

1262250519

VINAY BORSE

1262250653

PRATYUSH DEV

1262252536

RAGHAV SHELKE

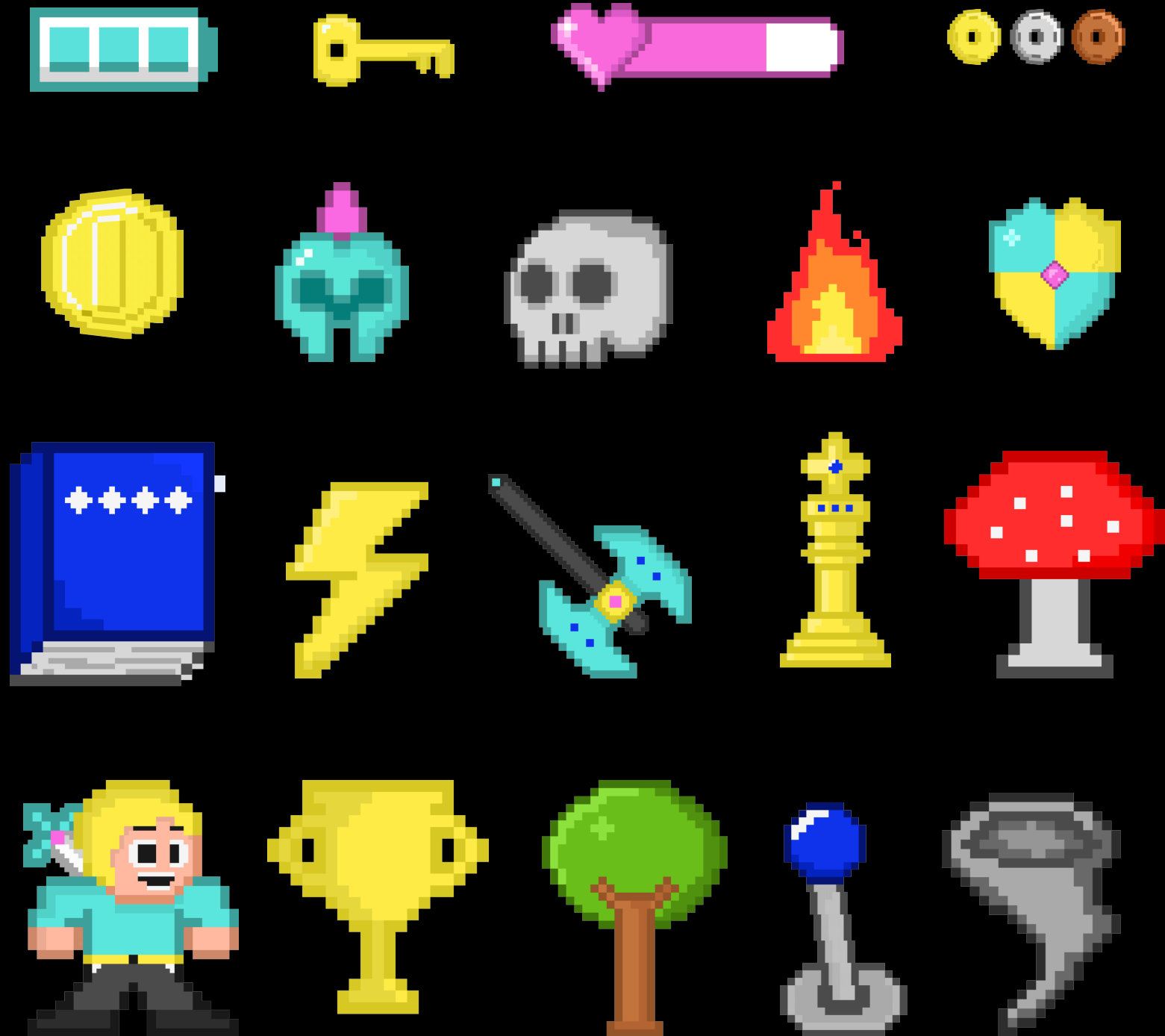
1262250833

JEET ZANVAR

1262250354

PROJECT OVERVIEW

BUILDING A CENTRALIZED
GAMING STATION USING THE
POWER OF C STANDARD
LIBRARIES. OUR GOAL WAS TO
BLEND PROCEDURAL PROGRAMMING
WITH ENGAGING USER
INTERACTION.

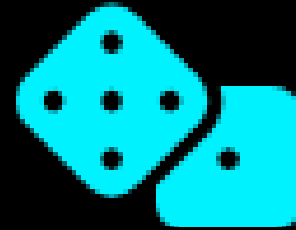


CORE DEVELOPMENT GOALS



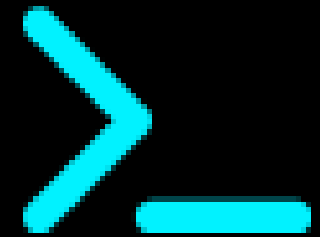
LOGIC MASTERY

IMPLEMENTING CLEAN CONTROL FLOWS USING SWITCH-CASES AND IF-ELSE STRUCTURES FOR RELIABLE GAMEPLAY.



RANDOMIZATION

UTILIZING TIME-SEEDED RANDOM GENERATION TO ENSURE EVERY GAME SESSION IS UNIQUE AND CHALLENGING.



CLI INTERACTION

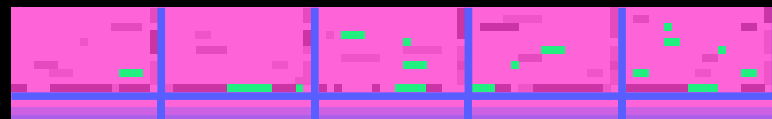
DESIGNING A ROBUST MENU SYSTEM THAT HANDLES USER INPUTS AND PREVENTS PROGRAM CRASHES.

MENU

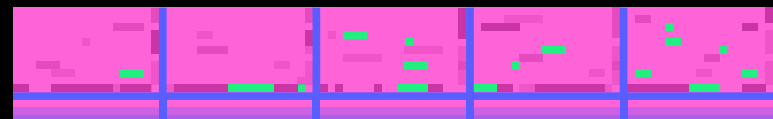
🗡️ 01 💎 07 ⭐ 12



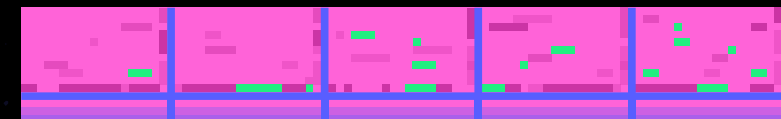
GAMES ROSTER



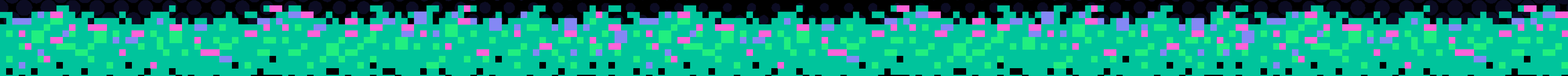
GAME 1
NUMBER GUESSING
GAME



GAME 2
ROCK, PAPER,
SCISSORS



GAME 3
GUESS THE WORD



GAME 1: NUMBER GUESSING

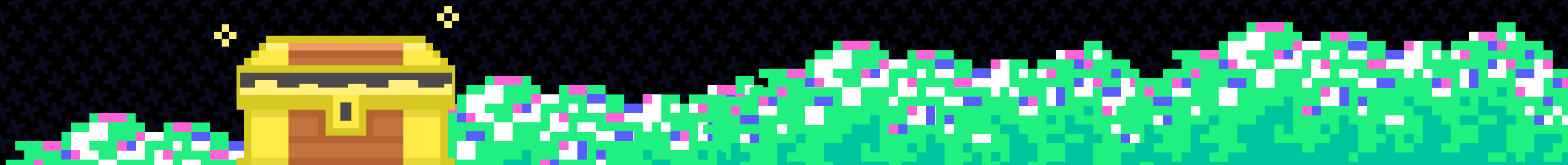
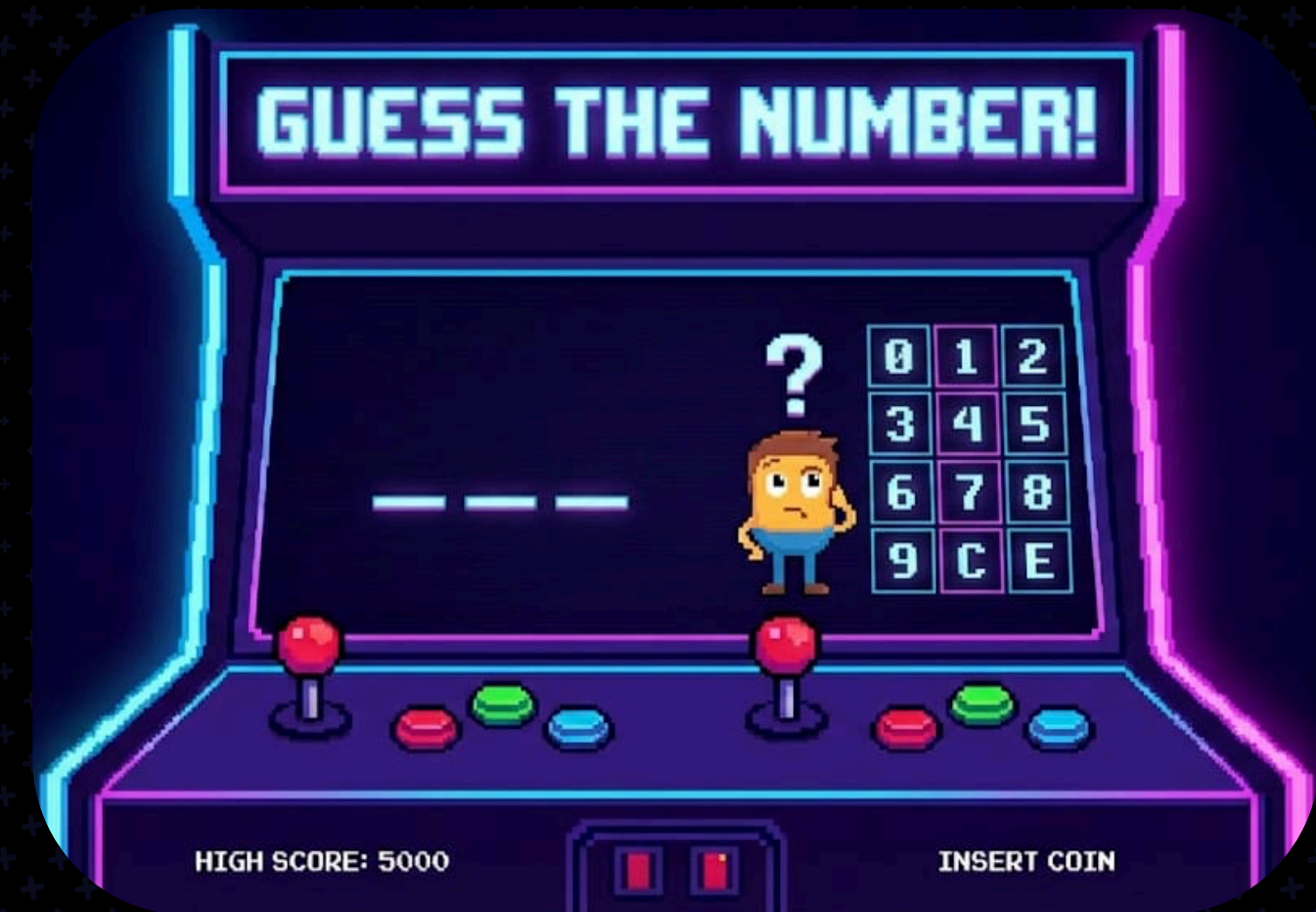
LOGIC OF NUMBER GUESSING

THE GAME GENERATES A HIDDEN INTEGER AND GUIDES THE USER THROUGH "HIGHER" OR "LOWER" PROMPTS.

DYNAMIC TRIAL TRACKING

INPUT VALIDATION FOR NON-NUMERIC ENTRIES

MATHEMATICAL RANGE RESTRICTION [1-100]



MENU

🔑 01

💎 07

★ 12



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <time.h>
5 #include <ctype.h>
6 |
7 void clearInputBuffer() {
8     int c;
9     while ((c = getchar()) != '\n' && c != EOF);
10 }
11
12 void playNumberGuess() {
13     int secret = (rand() % 100) + 1, guess, tries = 0;
14     printf("\n[Number Guess] I'm thinking of 1-100.\n");
15
16     do {
17         printf("Your guess: ");
18         if (scanf("%d", &guess) != 1) {
19             printf("Please enter a number!\n");
20             clearInputBuffer();
21             continue;
22         }
23         tries++;
24         if (guess > secret) printf("lower!\n");
25         else if (guess < secret) printf("higher!\n");
26     } while (guess != secret);
27
28     printf("Bingo! You got it in %d tries.\n", tries);
29 }
```

=====

THE C GAME HUB

=====

1. Number Guessing
2. Rock Paper Scissors
3. Guess the Word
4. Exit

Pick a game: 1

[Number Guess] I'm thinking of 1-100.

Your guess: 50

higher!

Your guess: 90

lower!

Your guess: 70

higher!

Your guess: 80

Bingo! You got it in 4 tries.

◆ GAME 2: ROCK PAPER SCISSORS

DECISION ENGINE

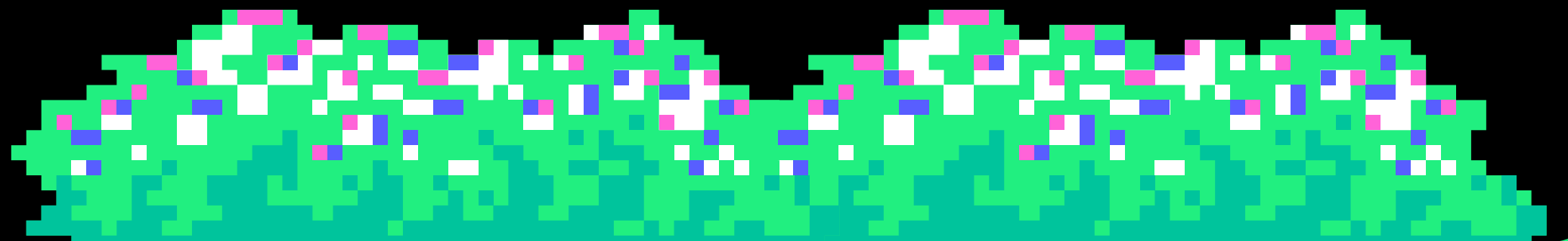
A CLASSIC TEST OF LUCK AND PROBABILITY. THE PROGRAM TRANSLATES NUMERICAL INPUTS (1-3) INTO GAME SYMBOLS AND CALCULATES WIN CONDITIONS INSTANTLY.

USES A MULTI-PATH CONDITIONAL LOGIC TO COMPARE USER SELECTIONS AGAINST COMPUTER-GENERATED RANDOM MOVES.

MENU

START

EXIT



```

void playRPS() {
    int user, computer = (rand() % 3) + 1;
    printf("\n[RPS] 1:Rock, 2:Paper, 3:Scissors: ");

    if (scanf("%d", &user) != 1) {
        printf("Invalid input.\n");
        clearInputBuffer();
        return;
    }

    if (user < 1 || user > 3) {
        printf("Invalid choice! Pick 1, 2, or 3.\n");
        return;
    }

    char *choices[] = {"", "Rock", "Paper", "Scissors"};
    printf("You: %s | Computer: %s\n", choices[user], choices[computer]);

    if (user == computer) printf("Draw!\n");
    else if ((user == 1 && computer == 3) ||
             (user == 2 && computer == 1) ||
             (user == 3 && computer == 2))
        printf("You Win!\n");
    else
        printf("Computer Wins!\n");
}

```

```

=====
THE C GAME HUB
=====

```

1. Number Guessing
2. Rock Paper Scissors
3. Guess the Word
4. Exit

Pick a game: 2

```

[RPS] 1:Rock, 2:Paper, 3:Scissors: 2
You: Paper | Computer: Paper
Draw!

```

```

=====
THE C GAME HUB
=====

```

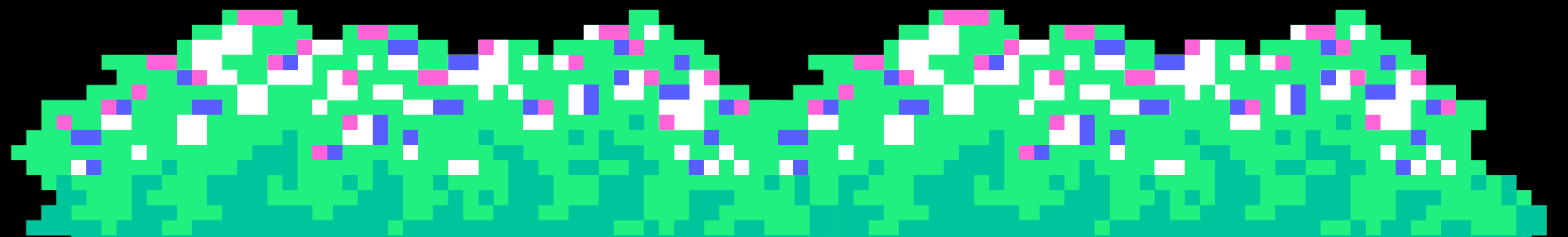
1. Number Guessing
2. Rock Paper Scissors
3. Guess the Word
4. Exit

Pick a game: 2

```

[RPS] 1:Rock, 2:Paper, 3:Scissors: 1
You: Rock | Computer: Scissors
You Win!

```



GAME 3: GUESS THE WORD



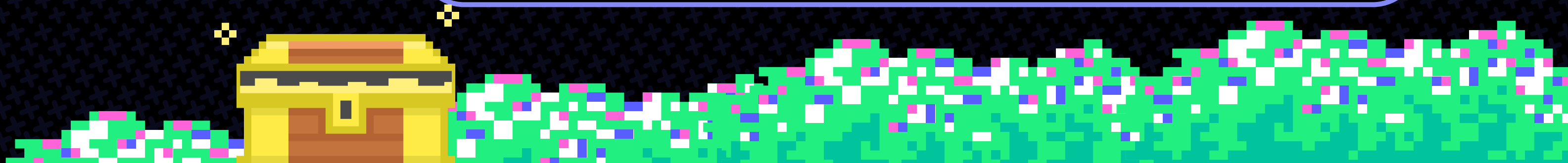
STRING PROCESSING

UNLIKE SIMPLE NUMBERS, THIS GAME HANDLES ARRAYS OF CHARACTERS AND CASE-INSENSITIVE MATCHING.

CONTEXT-AWARE HINTS FOR EVERY WORD

AUTOMATIC LOWERCASE CONVERSION

MEMORY-SAFE STRING COMPARISON



```

void playGuessWord() {
    char *words[] = {
        "apple", "banana", "orange", "grape", "mango",
        "cherry", "peach", "lemon", "berry", "melon"
    };

    char *hints[] = {
        "A common red fruit",
        "A long yellow fruit",
        "A citrus fruit that's also a color",
        "Small fruits that grow in bunches",
        "A tropical fruit with orange flesh",
        "Small red fruits used on desserts",
        "A fuzzy fruit with a pit",
        "A sour yellow citrus fruit",
        "A small sweet fruit (generic)",
        "Large fruit with green rind"
    };

    int count = sizeof(words) / sizeof(words[0]);
    int index = rand() % count;

    char guess[20];
    printf("\n[Guess the Word]\nHint: %s\n", hints[index]);
    printf("Enter word: ");
    scanf("%19s", guess);

    for (int i = 0; guess[i]; i++)
        guess[i] = tolower(guess[i]);

    if (strcmp(guess, words[index]) == 0)
        printf("Correct! 🎉 It was %s.\n", words[index]);
    else
        printf("Wrong! The answer was %s.\n", words[index]);

    clearInputBuffer();
}

```

Pick a game: 3

[Guess the Word]

Hint: Small fruits that grow in bunches

Enter word: grape

Correct! 🎉 It was grape.

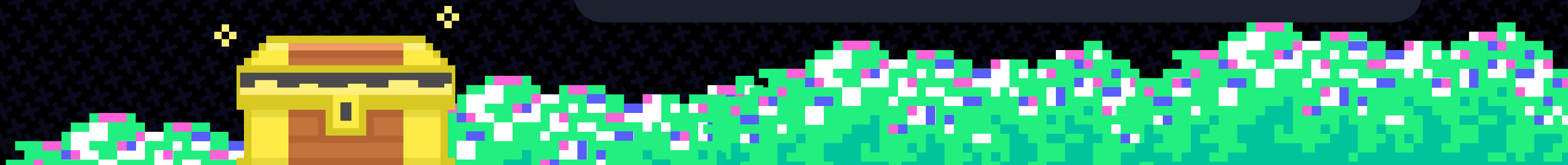
Pick a game: 3

[Guess the Word]

Hint: A sour yellow citrus fruit

Enter word: mango

Wrong! The answer was lemon.



MENU

🗝️ 01

💎 07

★ 12



```
int main() {
    int choice;
    srand(time(NULL));

    while (1) {
        printf("\n=====");
        printf("\n  THE C GAME HUB");
        printf("\n=====");
        printf("\n1. Number Guessing");
        printf("\n2. Rock Paper Scissors");
        printf("\n3. Guess the Word");
        printf("\n4. Exit");
        printf("\nPick a game: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Enter a number.\n");
            clearInputBuffer();
            continue;
        }

        if (choice == 4) break;

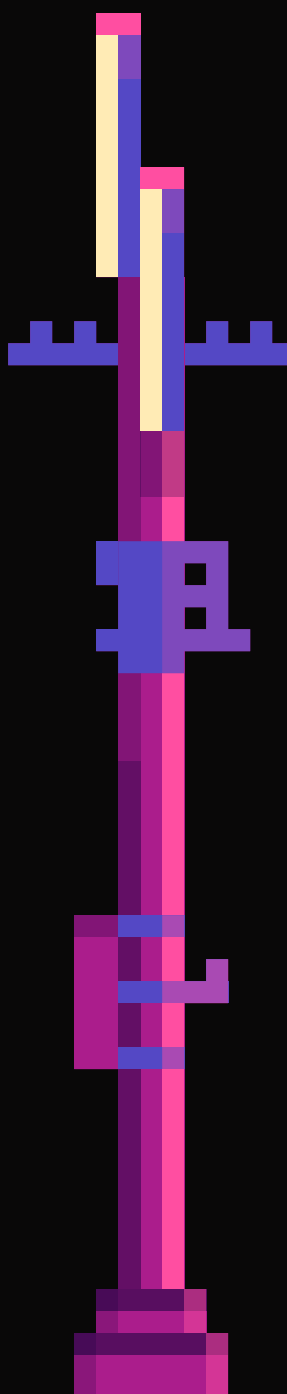
        switch (choice) {
            case 1: playNumberGuess(); break;
            case 2: playRPS(); break;
            case 3: playGuessWord(); break;
            default: printf("Try again (1-4).\n");
        }
    }

    printf("Goodbye!\n");
    return 0;
}
```





KEY LEARNINGS AND GROWTH



USER INPUT
HANDLING:
MASTERING SCANF
AND HANDLING NON-
INTEGER TRAPS.



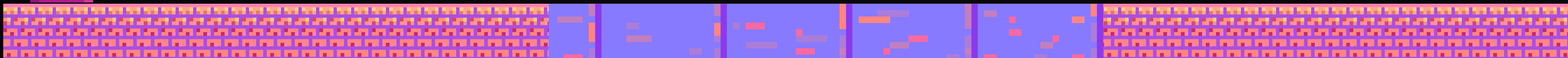
MODULAR PROGRAMMING:
SEPARATING GAME
LOGIC INTO DEDICATED
FUNCTIONS FOR BETTER
READABILITY.



RANDOMNESS:
IMPLEMENTING
SRAND(TIME(NULL))
FOR TRUE-RANDOM
GAMING FEEL.



CASE SENSITIVITY:
USING TOLOWER()
TO CREATE A
FORGIVING USER
EXPERIENCE.



POTENTIAL EXTENSIONS

- 🎮 UPGRADE FROM CLI TO GRAPHICAL USER INTERFACE (GUI)
- 👤 ADD MULTIPLAYER MODE (LOCAL / ONLINE)
- 🤖 IMPLEMENT ADVANCED AI & DIFFICULTY LEVELS
- 💾 INTRODUCE FILE HANDLING FOR HIGH SCORES & PLAYER PROFILES
- 📍 EXPAND GAME LIBRARY (TIC TAC TOE, SNAKE, QUIZ, ETC.)
- 🌐 ENSURE CROSS-PLATFORM COMPATIBILITY
- 🔊 ADD SOUND EFFECTS & BACKGROUND MUSIC
- 🏆 CREATE A LEADERBOARD & PERFORMANCE TRACKING SYSTEM

MENU



THANK YOU!

